

Programmation Pratique en Python

Projet Sherlock

Collecte et analyse de traces de mobilité dans le
cadre d'une « investigation »

Prof. Kévin Huguenin

INFORMATIONS GÉNÉRALES

- Groupes de 1 ou 2 (**maximum**) étudiant·e·s
 - Déclarer les groupes sur Moodle (délai : **mercredi 8 avril à 18h**)
 - Ne pas former de paire redoublant·e – nouvel·le étudiant·e (sauf si aucun rendu l'an dernier)
 - ⚠ Code trop similaire entre groupes $\Rightarrow$ 0 + rapport
- Séances du mercredi :
 - Début cette semaine : ~20 heures de travail encadré + travail personnel
- Rendu le **mercredi 3 juin à 18h** (⚠ pas d'extension):
 - **Code source** (relu et évalué. **Critères** : est-ce que l'idée est là, est-ce que le code marche, est-ce que le code a du sens, est-ce que le code est élégant et robuste)
 - **Feuille d'auto-évaluation** listant les tâches complètement faites/partiellement faites/non faites + commentaires
 - Un fichier zip contenant tous les fichiers .py (pas de sous répertoire) et la feuille d'auto-évaluation, nommé nom.zip ou nom1_nom2.zip (pour les groupes de deux étudiant·e·s), sans majuscule. p. ex. : huguenin.zip
 - Rendu par Moodle
- Modalité d'évaluation :
 - Bonus de 0, 0.25, 0.5 ou **0.75** points pour la note finale
 - Préparation pour **l'examen** : certaines questions de l'examen seront proches de ce qui est fait dans le projet
- ⚠ Pas de solution, pas de correction (juste une note et un commentaire)



PRÉSENTATION GÉNÉRALE



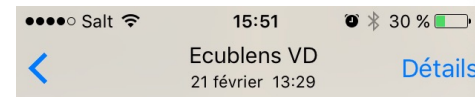
- Un **crime** a été commis : 📍 **lieu** et 🕒 **heure** !
- Identifie les suspect·e·s plausibles à partir de leurs traces de mobilité 📶
 - Plusieurs suspect·e·s
 - Traces de mobilité pour chaque suspect·e, issues de plusieurs sources incluant :
 - 📷 **Photos** géo-tagguées
 - 📶 **Logs** de connexion **Wi-Fi**
 - ☰ **Logs système** d'un **smartphone**
- Méthodologie générale [*disclaimer* : ce n'est **pas** une méthode d'investigation rigoureuse]
 1. Récupérer les différents **paramètres** et données fournis par les enquêteur·trice·s
 2. Construire les **traces de mobilité** des suspect·e·s
 3. Etablir la **plausibilité** qu'un·e suspect·e se soit rendu sur le **lieu de crime** à l'**heure du crime**
 - Extraire les localisations de chaque suspect **juste avant** et **juste après** le crime
 - Vérifier s'il est possible de se déplacer du crime à ces localisations dans le **temps imparti**



PHOTOS

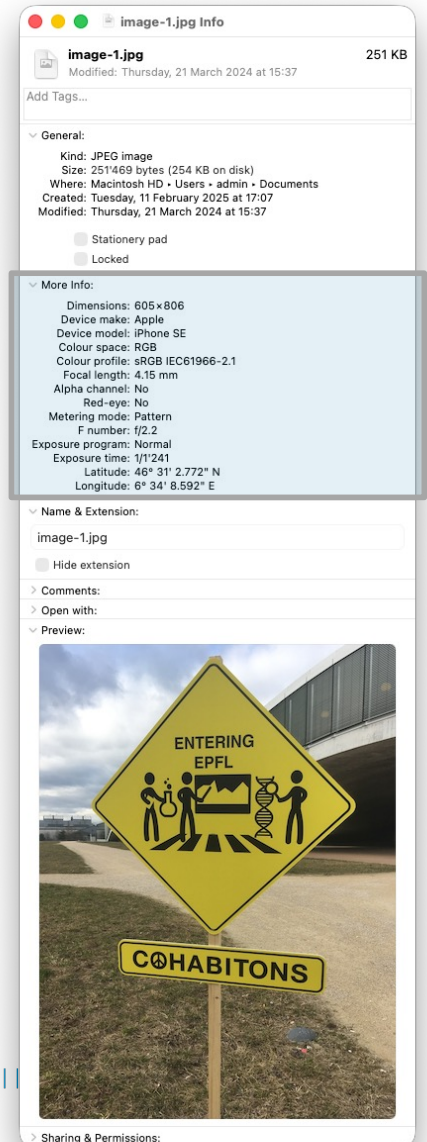
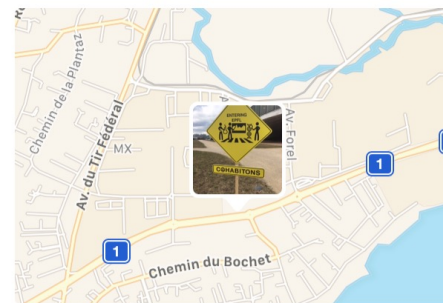
- Pour certain·e·s suspect·e·s, les enquêteur·trice·s ont retrouvé sur leurs disques durs certaines de leurs photos (prises avec un téléphone portable) qui contiennent les **dates** et la **localisation** auxquelles les photos ont été prises (données EXIF)

https://fr.wikipedia.org/wiki/Exchangeable_image_file_format



Lieux
Route
Cantonale
Ecublens VD
Suisse

Afficher les photos à proximité



WI-FI

- Les enquêteur·trice·s ont obtenu accès aux **logs de connexion du réseau Wi-Fi** de l'UNIL (via RADIUS/EAP). À chaque fois qu'un téléphone portable se connecte au réseau eduroam (il se connecte automatiquement dès lors qu'un point d'accès est à portée s'il est configuré pour utiliser ce réseau), un événement est créé dans les logs.

L'événement contient **l'identifiant UNIL de l'utilisateur·trice** et **l'identifiant du point d'accès Wi-Fi**. La **localisation** de ces points d'accès est connue.

https://fr.wikipedia.org/wiki/Remote_Authentication_Dial-In_User_Service

https://fr.wikipedia.org/wiki/Extensible_Authentication_Protocol

https://en.wikipedia.org/wiki/Wireless_LAN#Roaming

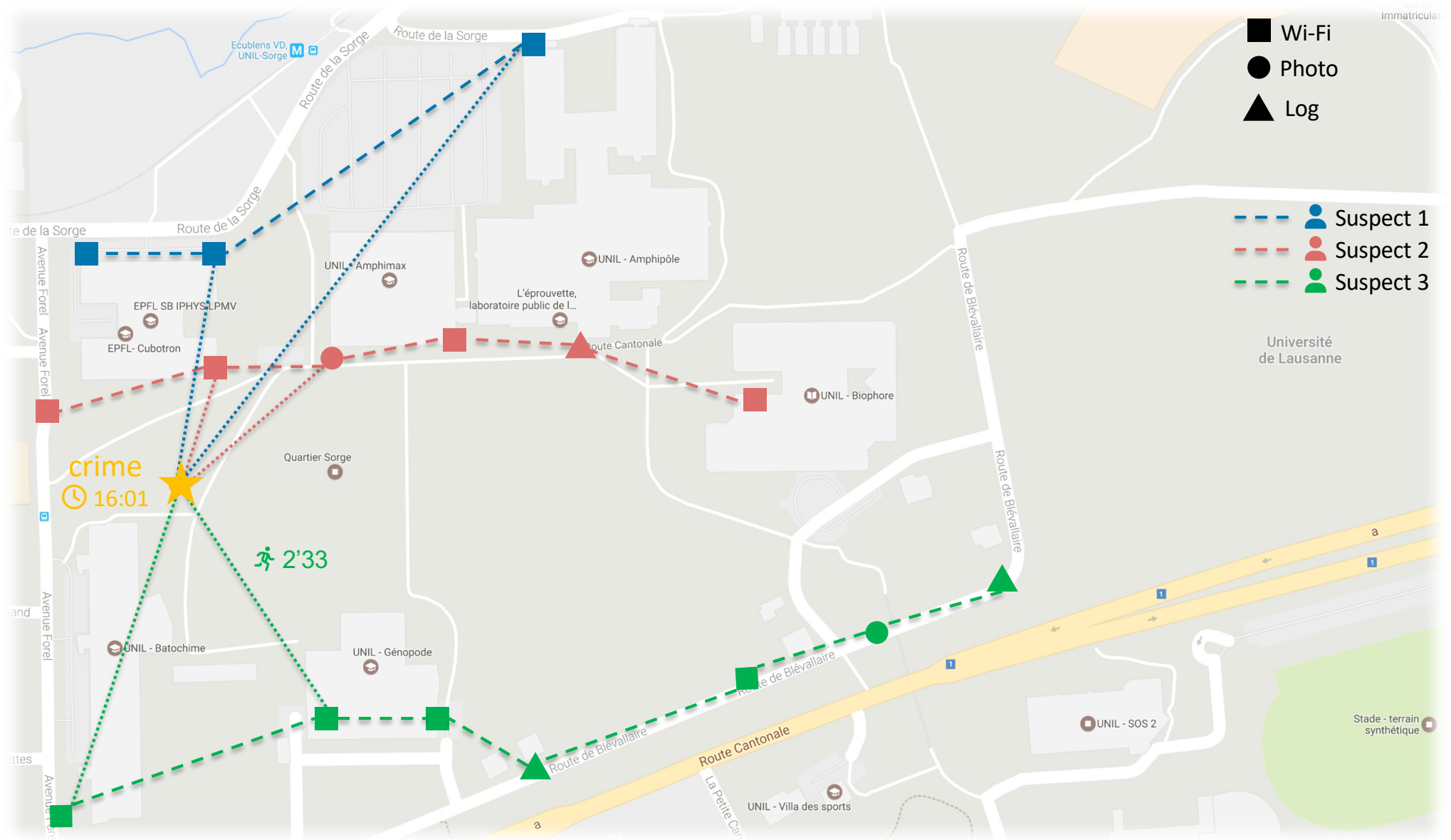
LOGS

- Les enquêteur·trice·s ont mis la main sur les fichiers de **log système** des téléphones portables des différent·e·s suspect·e·s. À chaque fois qu'une action est effectuée sur le smartphone, l'événement est consigné dans les logs du téléphone.

Chaque log contient la **date** exacte de sa création, et certains d'entre eux peuvent contenir une **localisation**.

```
[2026-03-21T13:08:17.716] Location request from app meteo, returned coordinates: UNKNOWN, source: GPS
[2026-03-21T13:08:19.462] notification received from app google maps, hash: (c1fb6c9f5164df2ceec382d7ac3b02a4a7a5cd6fe3ac8fc4f43dbad0117...)
[2026-03-21T13:08:21.523] notification received from app meteo, hash: (7f01f317aef0bc8060290b907a1517d19e5c49827de85e18ef0fe2494e2f40dd2...)
[2026-03-21T13:08:23.035] Location request from mobile cff, returned coordinates: (46.5216539599493, 6.5832874429498816), source: GPS
[2026-03-21T13:08:25.568] unplugging device, status: on battery, battery level: 97%
[2026-03-21T13:08:26.157] Location request from app mobile cff, returned coordinates: UNKNOWN, source: wifi
[2026-03-21T13:08:27.714] airplane mode enable
[2026-03-21T13:08:28.192] airplane mode disable
[2026-03-21T13:08:30.960] app firefox updated, current version: 148.3
[2026-03-21T13:08:32.325] notification received from app whatsapp, hash: (3db6606f2e529243babf0df178f781f43f5dc45af2b41803289af89c5ddd8e...)
[2026-03-21T13:08:33.042] app google maps updated, current version: 26.12.2
[2026-03-21T13:08:35.753] airplane mode enabled
[2026-03-21T13:08:37.948] airplane mode disabled
[2026-03-21T13:08:38.364] Location request from app agenda, returned coordinates: (47.2976323489746, 6.4632944378900276), source: cellular
...
```

💡 CONCEPT



DÉTAILS TECHNIQUES

> PARAMÈTRES

- Arguments de la ligne de commande (utiliser argparse) :

```
usage: sherlock.py [-h] [-v] -s SUSPECTS -g GEO_API_KEY -lat LATITUDE -lng LONGITUDE -d DATE
```

Identifie les suspect.e.s les plus plausibles à partir de leurs traces de mobilité (issues de sources multiples incluant les localisations contenues dans les traces Wi-Fi et les flux de photos géo-tagguées) pour un crime spécifié par une date/heure et une localisation.

options:

-h, --help	show this help message and exit
-v, --verbose	affiche les détails de l'exécution du programme et les avertissements.
-s, --suspects SUSPECTS	fichier contenant la liste des suspect.e.s et les sources de données de localisation.
-g, --geo-api-key GEO_API_KEY	clé pour l'accès à l'API du service de cartographie.
-lat, --latitude LATITUDE	latitude de la scène du crime.
-lng, --longitude LONGITUDE	longitude de la scène du crime.
-d, --date DATE	date et heure du crime (au format JJ/MM/AAAA-hh:mm, par exemple 17/03/2026-15:52:31).

DONNÉES

- Fichier de données : le fichier `suspects.xml` (utiliser `xml.etree.ElementTree`)

```
<?xml version="1.0"?>
<suspects>
  <suspect>
    <name>John</name>
    <sources>
      <source>
        <type>Wi-Fi</type>
        <db>db/wifi.db</db>
        <username>jdoe</username>
      </source>
      <source>
        <type>Photographs</type>
        <dir>pics/John</dir>
      </source>
      <source>
        <type>Logs</type>
        <file>logs/jdoe.log</file>
      </source>
    </sources>
  </suspect>
  ...
</suspects>
```

↗ Alternative : JSON

- Récupérer le **nom du fichier** depuis les arguments de la liste de commande (`argparse`), **analyser** le fichier XML (`xml.etree.ElementTree`), **créer les objets** correspondants (`LocationProvider`, `Suspect`).

SUSPECTS

- La classe `Suspect` décrit simplement des objets qui contiennent les informations décrites dans le fichier XML/JSON : un `nom` et un `location provider` (qui agrège plusieurs location providers, à l'aide du patron *Composite*)

© suspects.Suspect
(f) <code>_lp</code> (f) <code>_name</code> (f) <code>__repr__</code>
(m) <code>__init__(self, name: str, lp: LocationProvider)</code> (m) <code>get_name(self)</code> (m) <code>get_location_provider(self)</code> (m) <code>__str__(self)</code> (m) <code>create_suspects_from_xml_file(filename: str)</code> (m) <code>create_suspects_from_json_file(filename: str)</code>

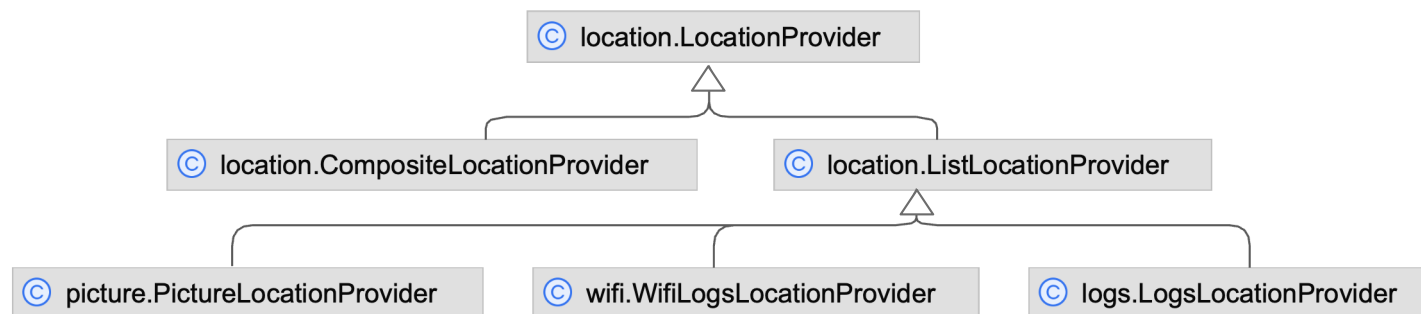
Powered by yFiles

Unil.

UNIL | Université de Lausanne

ARCHITECTURE

- Une classe `Location` et une classe `LocationSample`
 - Méthodes pour calculer des distances et trier par ordre chronologique
- Une classe *abstraite* `LocationProvider`
 - Spécifie / fournit les méthodes utiles
- Une classe *filie* `CompositeLocationProvider` appliquant le patron *Composite*
- Une classe *filie* `ListLocationProvider`
 - Gère une liste de `LocationSample`
- Une classe *filie* (héritant de `ListLocationProvider`) par type de source de données
 - Seules certaines méthodes sont (ré-)implémentées (redéfinies)



Powered by yUML

Unil.

UNIL | Université de Lausanne

📍 Location et LocationSample

- La classe `Location`
 - Décrit des objets contenant une latitude et une longitude...
 - ...initialisées dans `__init__` (vérifier les valeurs, lever une exception)...
 - ...accessibles via des *getters*
 - Contient une méthode pour calculer la distance entre deux objets [optionel]
- La classe `LocationSample`
 - Décrit des objets contenant un timestamp (datetime) et une `Location`...
 - ...initialisés dans `__init__` (vérifier les valeurs, lever une exception)...
 - ...accessibles via des *getters*
 - Surcharge les opérateurs de comparaison pour refléter l'ordre chronologique (`ls1 <= ls2` si `ls1` est antérieur à `ls2`). Il est donc possible d'utiliser `sort`, `min`, `max`, `<`, `>`

```

location.Location
├── _maps_adapter
├── _longitude
├── _latitude
├── _check_adapter_init(cls)
├── set_maps_adapter(cls, maps_adapter: MapsApiAdapter)
├── __init__(self, latitude: float, longitude: float)
├── __str__(self)
├── __repr__(self)
├── get_latitude(self)
├── get_longitude(self)
├── get_name(self)
├── get_travel_distance_and_time(self, destination: "Location", mode: str)

```

```

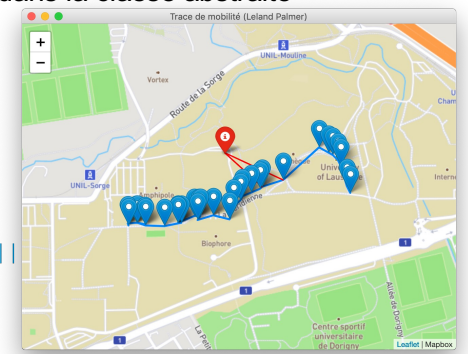
location.LocationSample
├── _location
├── _date
├── _description
├── __init__(self, date: datetime, location: Location, description: str)
├── get_location(self)
├── get_date(self)
├── get_description(self)
├── __str__(self)
├── __repr__(self)
├── __eq__(self, other: Any)
├── __ne__(self, other: Any)
├── __lt__(self, other: Any)
├── __le__(self, other: Any)
├── __gt__(self, other: Any)
├── __ge__(self, other: Any)

```


📍 LocationProvider

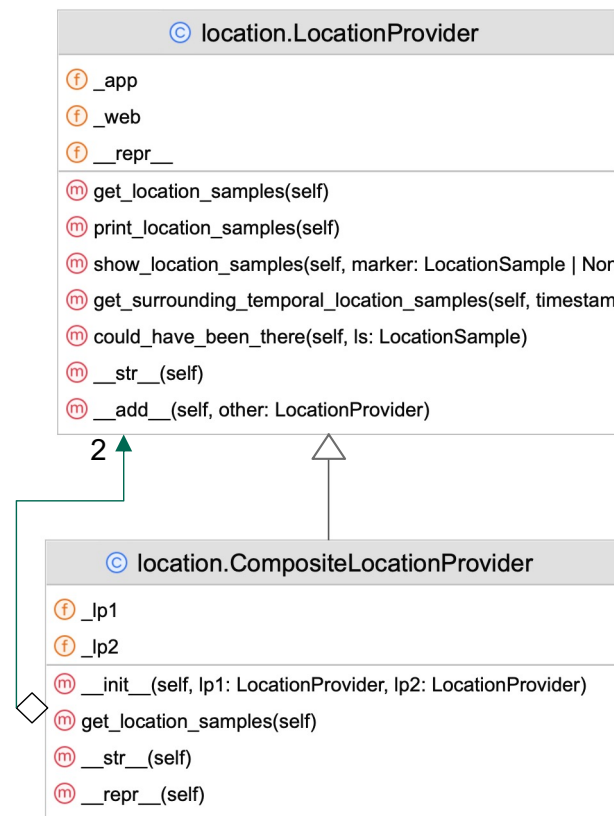
- La classe LocationProvider
 - Est abstraite (elle ne peut pas être instanciée)...
 - Spécifie l'existence d'une méthode `get_location_samples()` qui renvoie une liste d'objets LocationSample **triés par ordre chronologique**
 - Implémente un certain nombre de méthodes en utilisant `get_location_samples()` (implémentée par les classes filles)

© location.LocationProvider(ABC)	
(f) _app	
(f) _web	
(f) __repr__	
(m) get_location_samples(self)	Abstraite (implémentée par les classes filles)
(m) print_location_samples(self)	À implémenter dans la classe abstraite
(m) show_location_samples(self, marker: LocationSample = None, show: bool = True)	Fournie
(m) get_surrounding_temporal_location_samples(self, timestamp: date)	À implémenter dans la classe abstraite
(m) could_have_been_there(self, ls: LocationSample)	
(m) __str__(self)	
(m) __add__(self, other: "LocationProvider")	



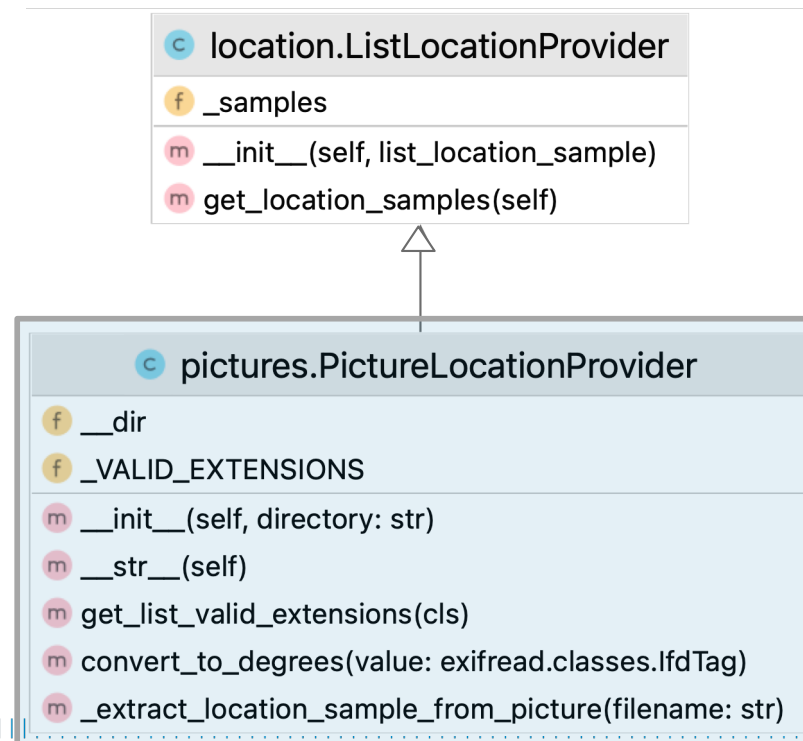
⊕ CompositeLocationProvider

- La classe CompositeLocationProvider
 - Hérite de la classe LocationProvider...
 - Implémente le patron Composite : Elle correspond à la composition de **deux** LocationProvider



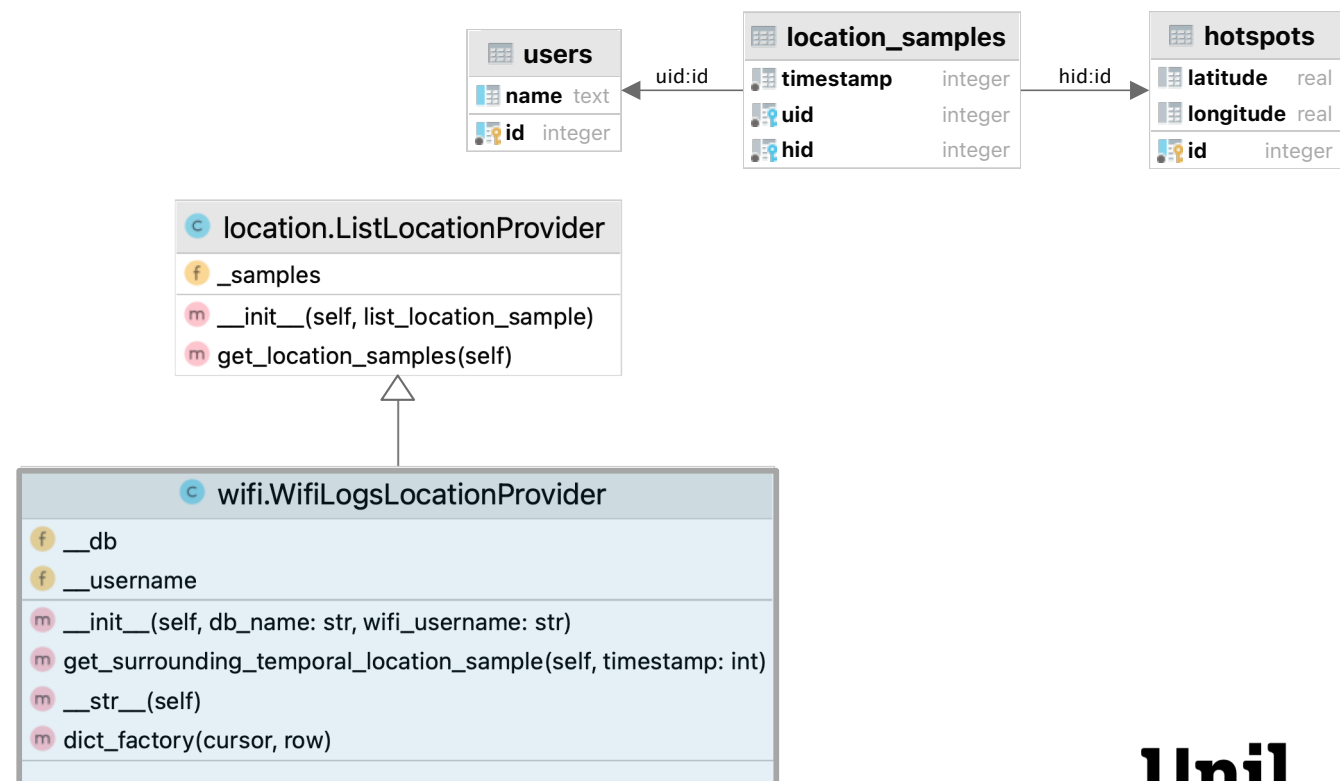
PictureLocationProvider

- La classe `PictureLocationProvider`
 - Hérite de la classe `ListLocationProvider`
 - Est construite à partir d'un répertoire (`dir`) passé en argument à `__init__`
 - Construit sa liste de `LocationSample` à partir de fichiers image au format JPEG (extension : `.jpg`, `.jpeg`, `.JPG`, `.JPEG`) contenu dans le répertoire `dir`
 - La date/heure et la localisation des photos sont extraites des informations EXIF (module `exifread`)



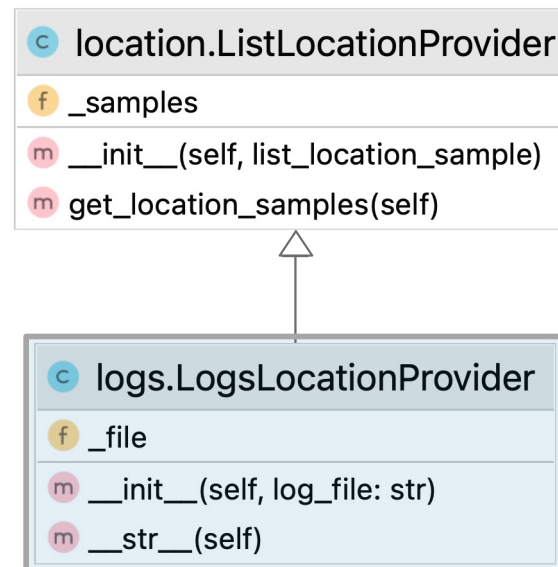
WifiLogsLocationProvider

- La classe WifiLogsLocationProvider
 - Hérite de la classe ListLocationProvider...
 - Est construite à partir d'un **nom d'utilisateur UNIL** et d'une base de données **SQLite** (module sqlite3)
 - La base de données a le format suivant:



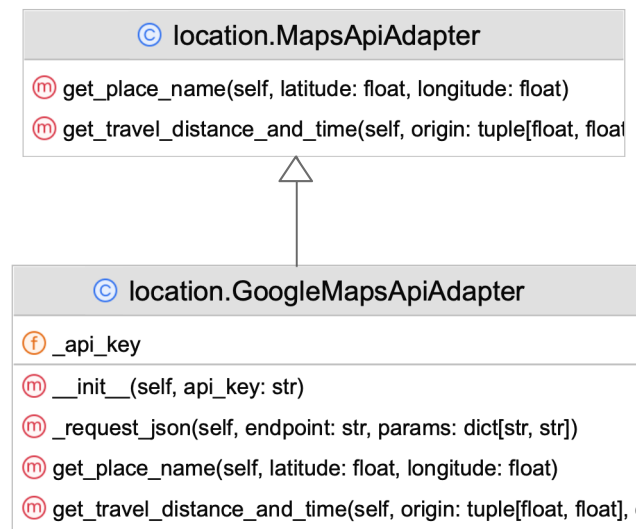
☰ LogsLocationProvider

- La classe LogsLocationProvider
 - Hérite de la classe ListLocationProvider
 - Est construite à partir d'un fichier (log_file) passé en argument à `__init__`
 - Construit sa liste de LocationSample à partir d'un fichier de logs (extension : .log)
 - La date/heure et la localisation sont extraites des logs à l'aide d'une [expression régulière](#)



CARTOGRAPHIE

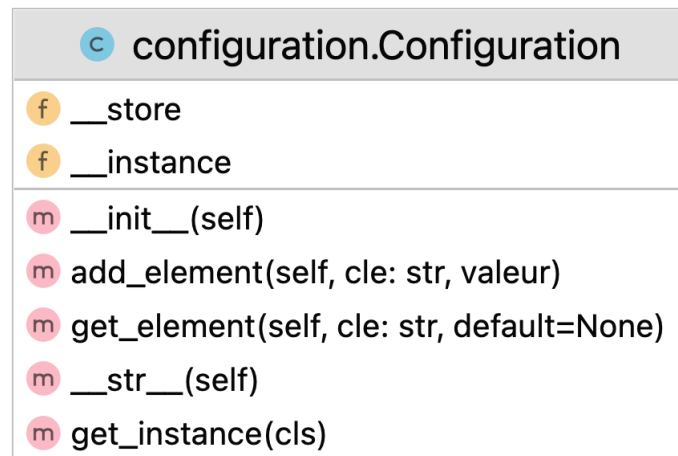
- Enrichir la classe `Location` en utilisant un objet `MapsApiAdapter` gérant l'accès à un API cartographique (module `urllib`):
 -  Une méthode pour obtenir une représentation textuelle de la localisation (p. ex. « Bâtiment Batochime, 1015 Lausanne, Suisse »)
 - API Reverse Geocoding
 -  Une méthode pour calculer la distance du trajet (à pied) d'une `Location` à une autre ( **attention** au sens)
 - API Distance Matrix
- Gérer la clé et envoyer des requêtes au niveau de la classe `GoogleMapsApiAdapter`





CONFIGURATION GLOBALE

- Les paramètres de configuration globale du programme (p. ex. verbose) sont réunis dans un objet de la Configuration, sous la forme d'un dictionnaire clé-valeurs ("verbose": True).
 - Initialisée dans le module principal
- La classe implémente le patron de conception Singleton pour permettre de manipuler la configuration globale de manière cohérente et facilement partout dans le programme
 - Utilisée dans les autres modules



DÉMARRAGE DU PROJET

-  Package pour démarrer (disponible sur Moodle) :

1. Instructions détaillées (document pdf) contenant

-  Des diagrammes
-  Des exemples
-  Des indices

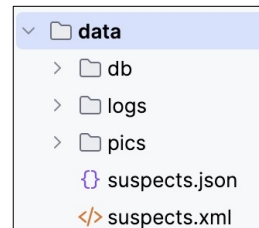
2. Le squelette du projet avec des TODOs (visibles dans PyCharm)

```
date = exif_data.get("GPS GPSTimeStamp")
timestamp = exif_data.get("GPS GPSTimeStamp")
# TODO: Convertir la date (au format textuel) contenue dans le EXIF tag en datetime et le stocker dans la variable t
```

3. Des méthodes de test dans le bloc main de chaque module avec le résultat attendu

```
# Tester l'implémentation de cette classe avec ces instructions (le résultat attendu est affiché ci-dessous)
# Configuration.get_instance().add_element("verbose", True)
# lp = PictureLocationProvider('../data/pics/jdoe')
# print(lp)
# Warning: Skipping file 'unil.jpg' (Missing time and/or location information)
# PictureLocationProvider(source: '../data/pics/jdoe'(JPG, JPEG, jpg, jpeg), 2 location samples)
# LocationSample[datetime: 2026-02-21 13:29:04, location: Location [latitude: 46.51744, longitude: 6.56905]]
# LocationSample[datetime: 2026-03-08 10:36:00, location: Location [latitude: 46.52206, longitude: 6.58417]]
```

4. Les données



DÉMARRAGE DU PROJET

- **?** Questions :
 - Pendant les séances dédiées
 - Sur Moodle
 - Par e-mail, titre « [esc-info2] projet », destinataires : professeur + assistant·e·s

CONCLUSION

❓ Questions



À suivre...